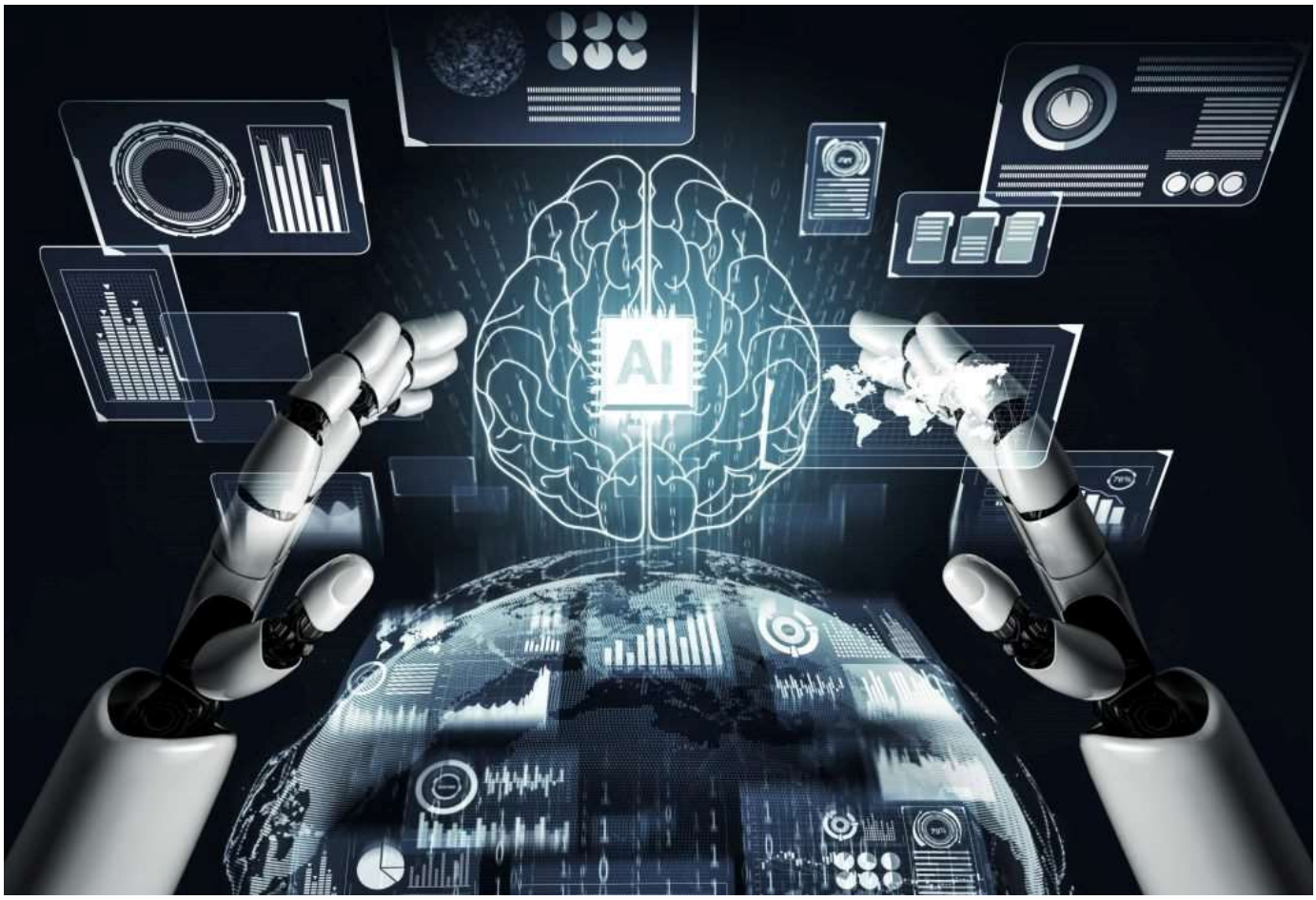
N-QUEENS



Artificial Intelligence

Contents:

Introduction and Overview

1. Project Idea:

The N-Queens project aims to design an implement a program capable of solving the N-Queen problem for different board sizes.

The value of N is selected by user, allowing the program to work with small and large boards alike.

To deal with this challenge, the project explores 4 different Ai search techniques:

- Backtracking
- Best-First
- Hill-Climbing
- Cultural

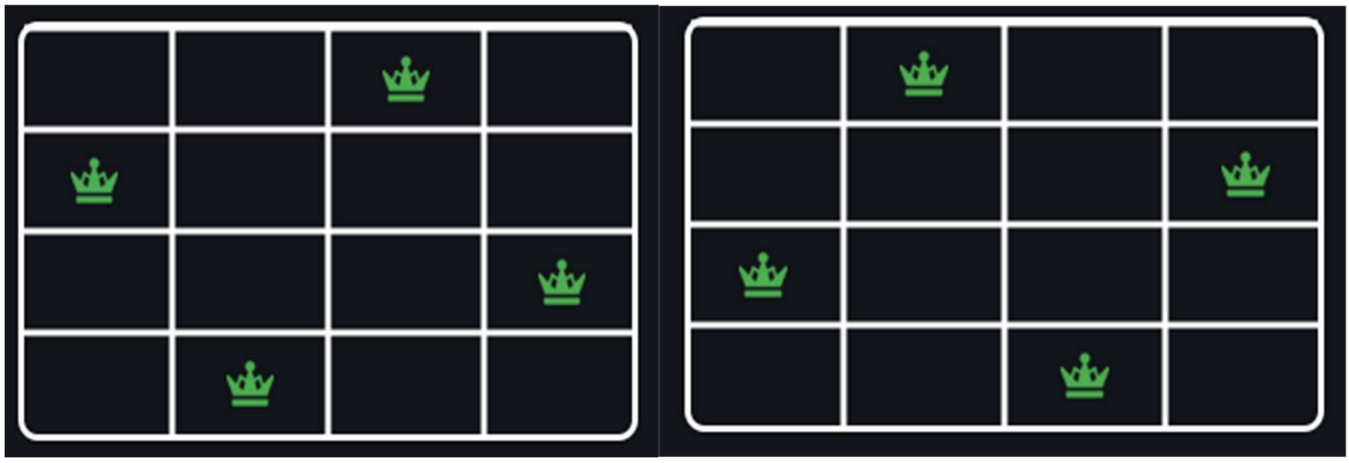
Using multiple approaches allow comparison of performance, effectiveness, and search behavior.

2. Problem Description:

In the N-Queen problem, the task is to arrange N queens on a NxN board so that no queen is attacking any other queen.

So to satisfy that **no** 2 queens must share the same row, column, or diagonal

As N increases, the number of possible board states grows exponentially, making the problem good for comparing search algorithms



Possible solutions for 4-Queens

Proposed Solution & System Features

1. Main Responsibilities:

The N-Queens program provides a set of core functionalities that allow user to explore and compare different algorithms for solving the N-Queens problem

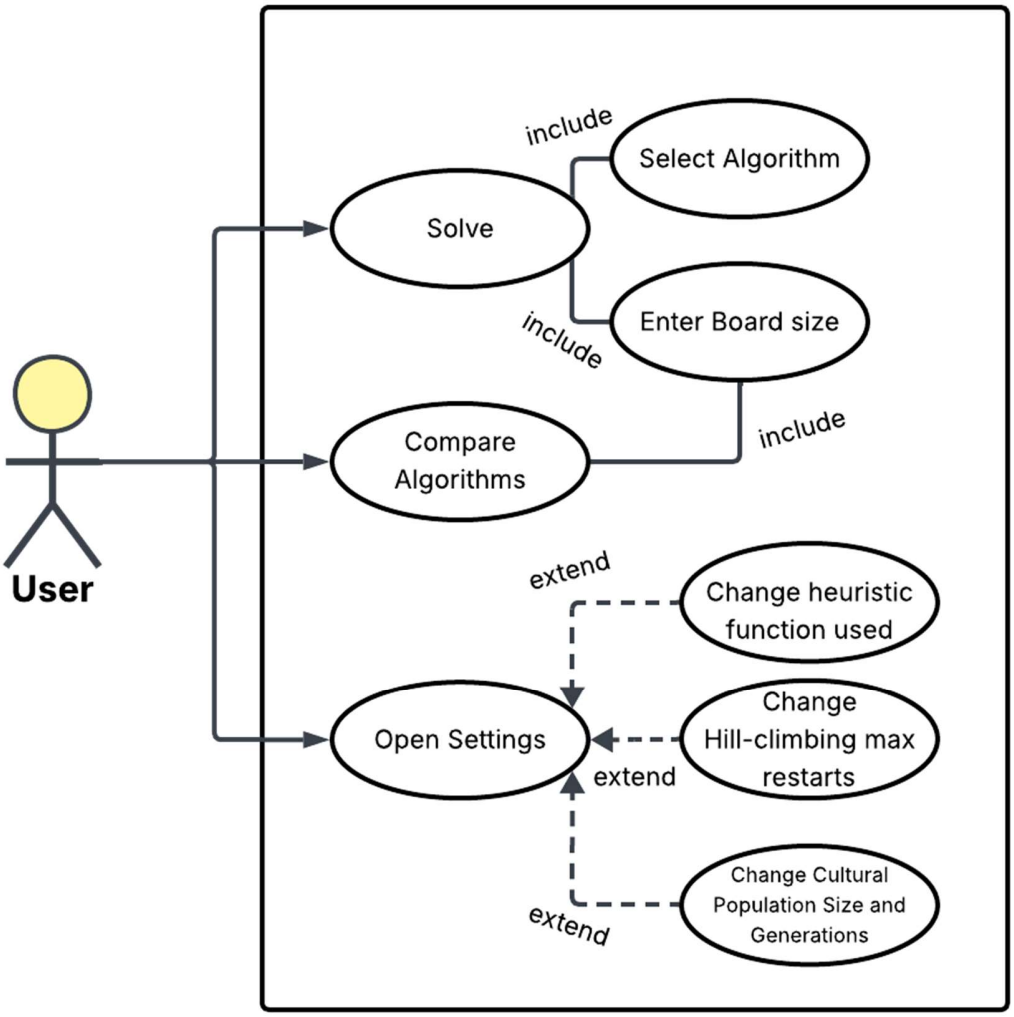
- User Input for Board Size:
    - Allow user to enter value of N which determine board size
    - Validates input
- Solving using the 4 algorithms individually:
    - **Backtracking**: Systematically explores all valid queen placements

    - **Best-First**: Uses heuristic function to guide search by selecting the most promising path first and aiming to find valid solution faster by reducing amount of blind exploration
    - **Hill-Climbing**: Start with initial state and iteratively improves by reducing conflicts, it also preforms local moves to reach better states
    - **Cultural**: Uses evolutionary strategies combined with belief space, it updates both population and belief space to accelerate convergence and generate improved candidate solutions over multiple generations

- User Interface:
    - Display board showing position of queens

- Allow user to view solution of selected algorithm or all of them
- Display time taken to find solution
- Allow user to compare all algorithms
- Show plot comparison of Algorithms

2. Use-Case:



3. Development Tools:

- Programming Language: Python
- IDE: PyCharm, VSC
- Libraries: flet->GUI
  heapq->Priority queue
  random->Random initials
  time->Record time taken
  Matplot->Plots
- Version Control: GitHub

Applied Algorithms

1. Backtracking:

Backtracking is a **systemic search technique** that tries to reach solution step by step and **backtracks** choices when they lead to a conflict

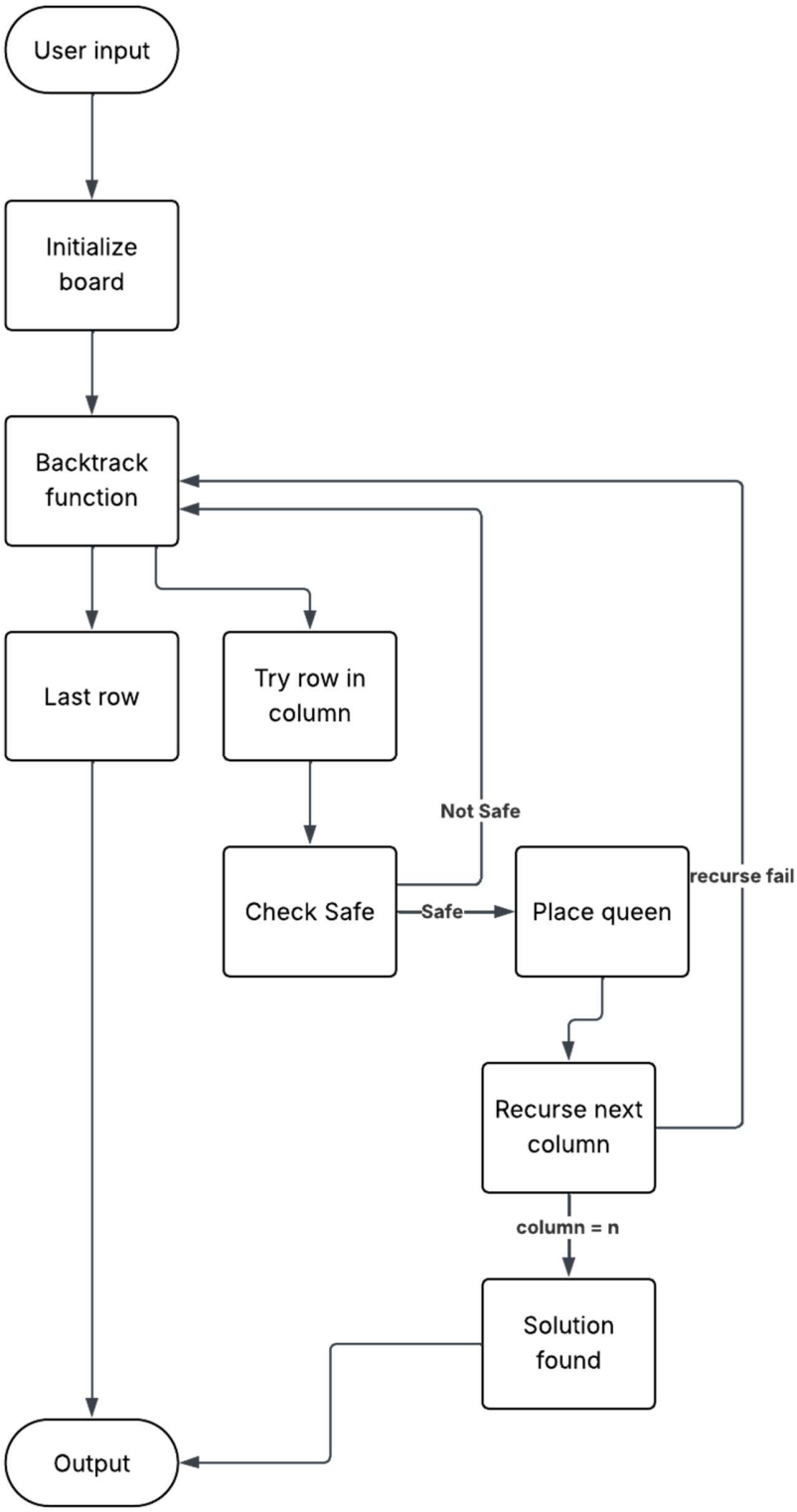
In N-Queens, it places 1 queen per column and explore all possible placements

- When board is empty start at first column
- Try placing a queen in a row, if row 0 cause conflict, try row 1 and if it also causes conflict try row 2 and etc. until a safe move is found
- If safe move is found, place queen in row and move to next column (Recursive call)
- If safe move isn't found, backtrack by returning to previous column and

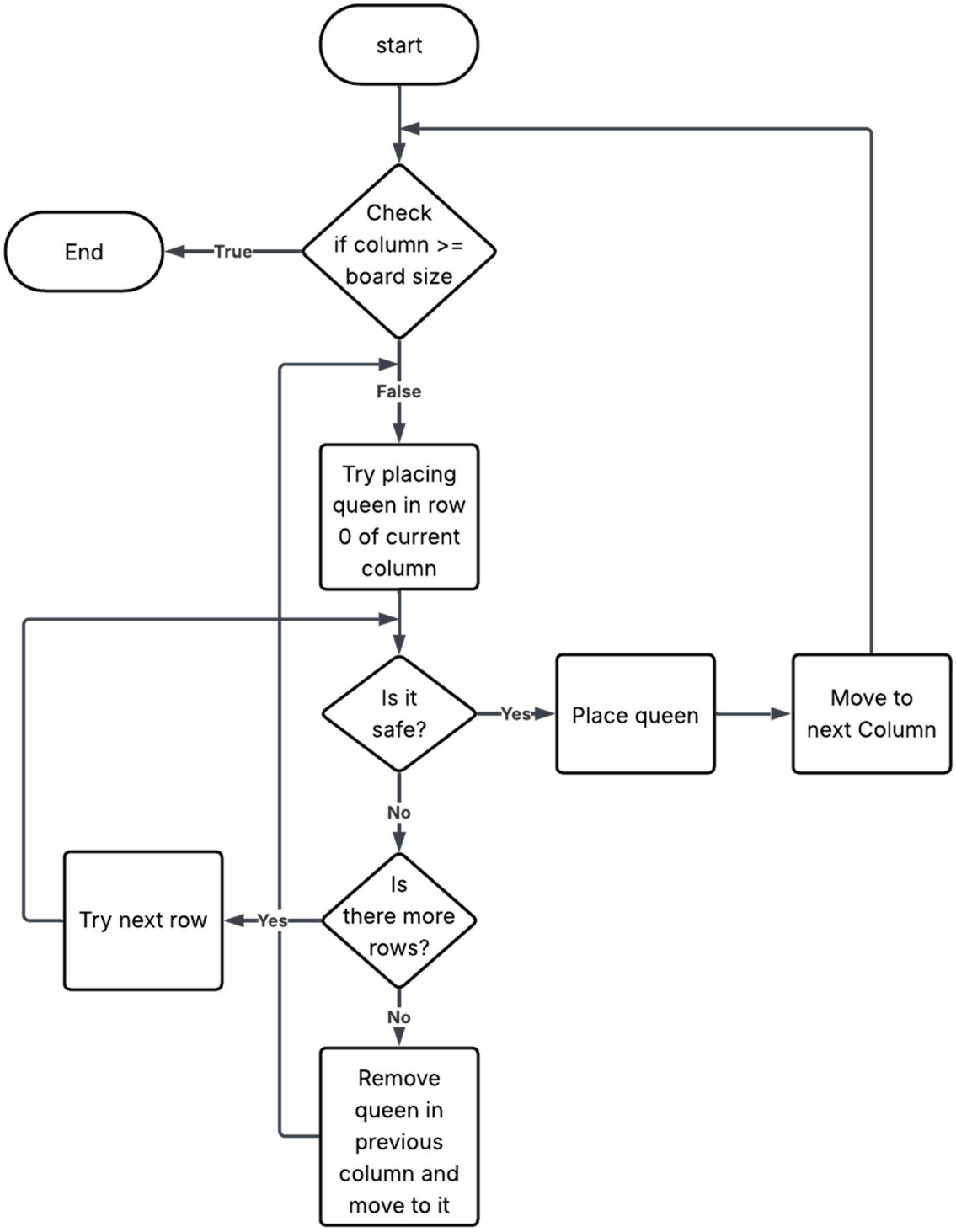
remove the queen there and try the next row in that old column

- Continue doing this process until column == N which would means that N queens have been placed and a solution has been found
- Algorithm ends when it finds solution or tried all valid ways of placing queen
- **Backtracking guarantees correct solution, avoid exploring invalid branches, and efficient for small values of N**

Block Diagram:



Flowchart:


start
Check if column >= board size
True
End
False
Try placing queen in row 0 of current column
Is it safe?
Yes
Place queen
Move to next Column
No
Is there more rows?
Yes
Try next row
No
Remove queen in previous column and move to it

- ## Literature Review:

  Backtracking has long been a core method for solving constraint satisfaction problems, and extensive research has examined both its theory and the factors behind its effectiveness. Although many variants exist, such as Forward Checking, Backjumping, and Conflict-Directed Backjumping. The papers shows that they can all be understood through a shared set of underlying principles. Two major research directions shaped this modern view: a unified interpretation of backtracking based on no-goods, and formal analyses of the search trees and consistency checks that different algorithms necessarily explore.

- A central insight is that these algorithms differ not in their fundamental operations, but in how they record and exploit information about inconsistent partial assignments. The concept of a no-good an assignment combination that cannot appear in any solution, provides a common framework for understanding all backtracking methods. Within this perspective, forward-looking techniques and backward-looking techniques are simply different strategies for discovering, storing, and reusing no-goods. The pruning strength of an algorithm depends on the types of no-goods it learns, how detailed they are, and how long they are retained.
- The papers further clarify how these algorithms traverse the search space. Formal characterisations identify exactly which nodes must be visited or avoided by each method, grounding claims about pruning power. For instance, algorithms enforcing stronger consistency, such as Forward Checking, skip many nodes that simpler chronological backtracking must explore. Conflict-based methods go further by identifying the deeper sources of inconsistencies and skipping whole portions of the search tree.
- It also helped clarify correctness and complexity, especially for early conflict-directed methods like backjumping and its extensions, which originally lacked clear formal justification

- **References:**

[1] F. Bacchus, “A uniform view of backtracking,” *Principles and Practice of Constraint Programming (CP)*, pp. 232–247, 2001.

[2] G. Kondrak and P. van Beek, “A theoretical evaluation of selected backtracking algorithms,” *Artificial Intelligence*, vol. 89, no. 1–2, pp. 365–387, 1997.

2. Best-First:

Overview

1. The best_first function implements a **Best-First Search** strategy to solve the **N-Queens problem**.
2. It attempts to find a board configuration where no queens attack each other by repeatedly exploring the most promising states first, using a heuristic score as the priority.

The algorithm uses:

- A **priority queue** (heapq) to always expand the state with the lowest heuristic value.
- A **heuristic function** Heuristics.current to evaluate states.
- A **visited set** to avoid revisiting previously seen configurations.

Function Signature

```
def best_first (board):
```

Parameters

- **board**: An object representing the N-Queens board.
  - board.N: number of queens (and board size)
  - board.start: initial state – a list where state[col] = row of a queen
  - board.place_queen(row, col): places a queen on the final board

---

How the Algorithm Works :

1. Initialization

```
n = board.N
start = board.start.copy()
heuristic = Heuristics.current
pq = [(heuristic(start, n), start)]
visited = set()
```

- start is the initial configuration.
- The priority queue contains (heuristic_value, state).

- The lower the heuristic, the more promising the state.
- `visited` stores states already expanded.

---

2. Main Loop: Priority-Driven State Expansion

```
while pq:
    h, state = heapq.heappop(pq)
```

- The queue always pops the state with the **lowest heuristic**.
- `h` is the number of conflicts
- `state` is a list of queen positions

---

3. Goal Test

```
if h == 0:
    for col in range(n):
        board.place_queen(state[col], col)
    return True
```

- If heuristic = 0 → no queens attack each other → solution found.
- The queens are placed onto the board.

---

4. Mark State → Visited

```
visited.add(tuple(state))
```

- States are converted to tuples because lists are not hashable.
- Prevents infinite loops or redundant computation.

---

5. Generate Neighbor States

```
for i in range(n):
    for j in range(n):
        if j != state[i]:
            new_state = state.copy()
            new_state[i] = j
            ...
```

For each column `i`:

- Try placing the queen in every other row `j`
- Each new configuration is a neighbor state differing by one queen move.

---

6. Push Valid New States to the Priority Queue

```
if tuple(new_state) not in visited:
    heapq.heappush(pq, (heuristic(new_state, n), new_state))
```

- Only unvisited states are added.
- Priority is determined again by the heuristic value.

---

7. Failure Case

```
return False
```

If all states are exhausted and no solution has heuristic 0, the algorithm reports failure.

Algorithm Characteristics

Search Strategy

- **Greedy Best-First Search**: always expands the state with minimum heuristic.
- **Not guaranteed optimal**, but often fast.

Heuristic

- Depends on `Heuristics.current`.
- Usually→ number of attacking queen pairs.

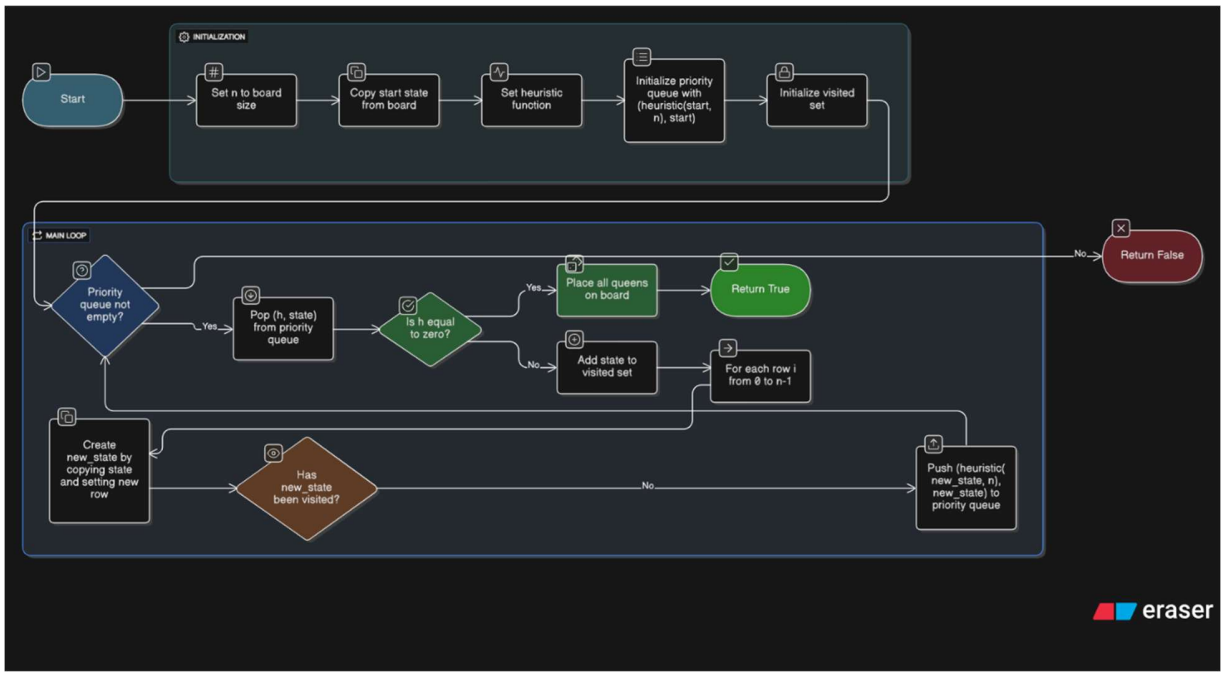
State Space

- Each state is an array of size `n`

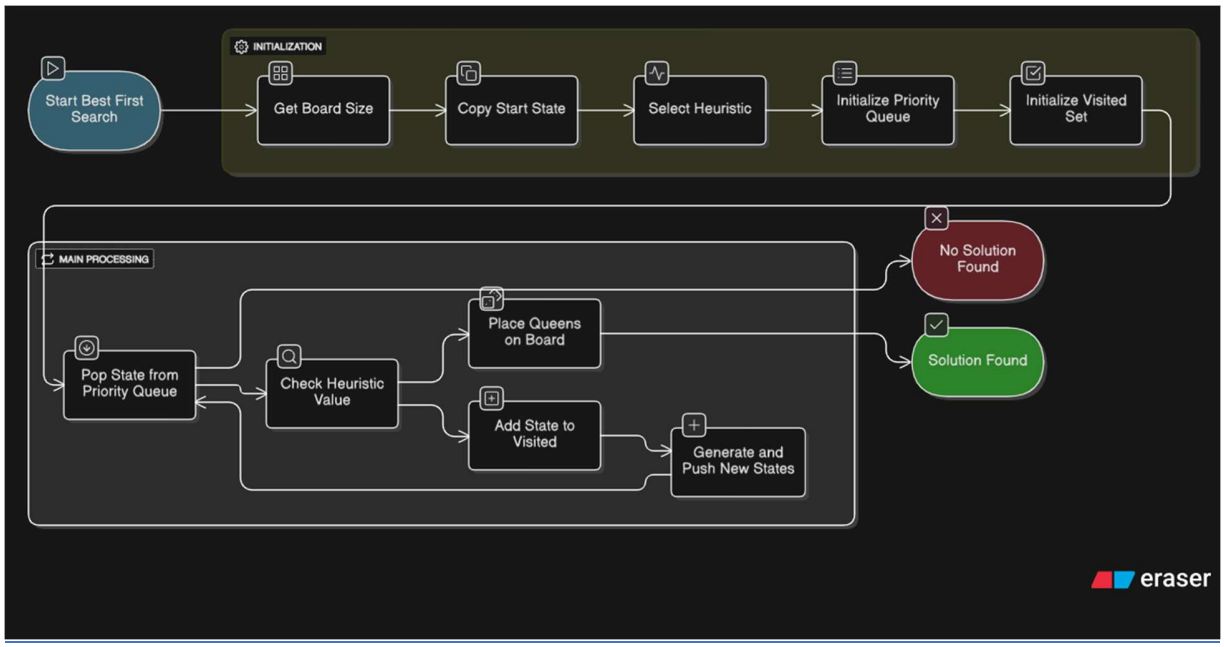
Complexity

- Worst-case: exponential (as with most N-Queens search algorithms)

Flowchart:



Block diagram:



Paper Summary

This paper generalizes the theory of **best-first search** by studying evaluation functions that depend on the *entire path* rather than only the standard A* components g(n)g(n)g(n) and h(n)h(n)h(n). The authors analyze how well-known properties of A*—such as admissibility, termination, node-expansion conditions, and optimality—extend to this broader class of **generalized best-first (GBF)** strategies.

Key Contributions

1. A general framework for path-dependent evaluation functions

The paper introduces a generalized formulation where the evaluation function may depend on the entire path, not just g(n)+h(n)g(n)+h(n)g(n)+h(n). This includes nonlinear, nonadditive, and path-dependent heuristics.

They show that many properties of A* extend naturally when fff is *order-preserving* (i.e., expanding a path cannot reverse earlier merit comparisons).

---

2. Termination, completeness, and solution-quality results

The authors define a critical value called **M** (a "minimax saddle value"), based on the maximum fff-value along solution paths.

They prove:

- **The algorithm always terminates** if fff is unbounded on infinite paths.
- **The returned solution has cost ≤ $\phi^{-1}(M)$** for monotonic fff.
  This produces bounds for **suboptimal but non-admissible heuristics**—including weighted A* variants and dynamically weighted schemes.

---

3. Generalized node-expansion conditions

They generalize the classic A* expansion condition:

f(n)≤C∗f(n) \le C^*f(n)≤C∗

to arbitrary GBF strategies. They provide:

- **Necessary and sufficient conditions** for when a node must be expanded.
- Specialized, exact conditions for **tree search**, independent of knowing the final solution path.

These conditions support performance analysis of GBF under broader classes of heuristics.

References Mentioned in the Paper

1. Bagchi, A., and Mahanti, A. *Search Algorithms Under Different Kinds of Heuristics.*
2. Dechter, R., and Pearl, J. *Generalized Best-First Search Strategies and the Optimality of A** (the present paper).
3. Bellman, R. *Dynamic Programming.*
4. Gelperin, D. *On the Optimality of A**.
5. Pohl, I. *Heuristic Search Viewed as Path Finding in a Graph.*
6. Hart, P. E., Nilsson, N. J., and Raphael, B. *A Formal Basis for the Heuristic Determination of Minimum Cost Paths.*
7. Hart, P. E., Nilsson, N. J., and Raphael, B. *Correction to "A Formal Basis..."*
8. Gaschnig, J. *Heuristic Search Algorithms.*
9. Karp, R. *Probabilistic Analysis of Search Algorithms* (relates to Appendix).
10. Lawler, E. L., and Wood, D. E. *Branch-and-Bound Methods.*
11. Pearl, J. *Heuristics: Intelligent Search Strategies for Computer Problem Solving.*
12. Miro, R. *On the Optimality of A**.
13. Nilsson, N. J. *Principles of Artificial Intelligence.*
14. Nilsson, N. J. *Problem-Solving Methods in Artificial Intelligence.*

3. Hill-Climbing:

The algorithm works iteratively, starting with a random or initial solution and constantly trying to improve it

1-Initialization:

The algorithm starts from a random point in the search space.

```
current = [random.randint(0, n - 1) for _ in range(n)]
```

2-Iteration and Improvement:

In each step, the algorithm:

- Generates all neighboring states (solutions) to the current state.

```
def get_neighbors(state):
```

- Evaluates all these neighboring solutions using a Heuristic Function, which measures the quality or "fitness" of the solution.
- Selects the best solution from the neighbors.

3-Movement:

If the best neighboring solution found is better than the current solution, the algorithm moves to this better solution, and it becomes the new current state.

```
next_state = min(neighbors, key=lambda s: heuristic(s, n)
```

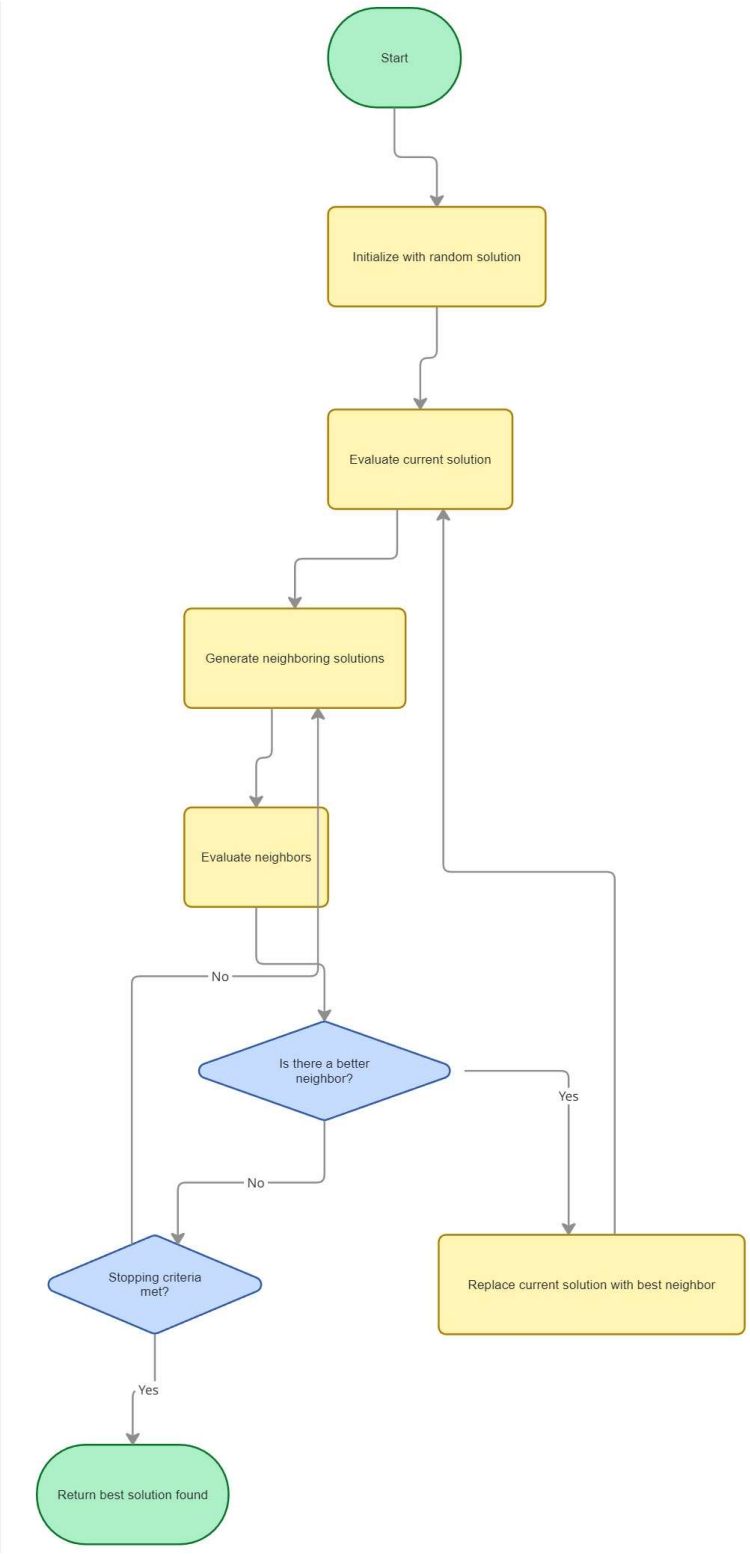
4- Termination:

The algorithm stops when no better neighboring solution can be found than the current state. This point is the Local Maximum.

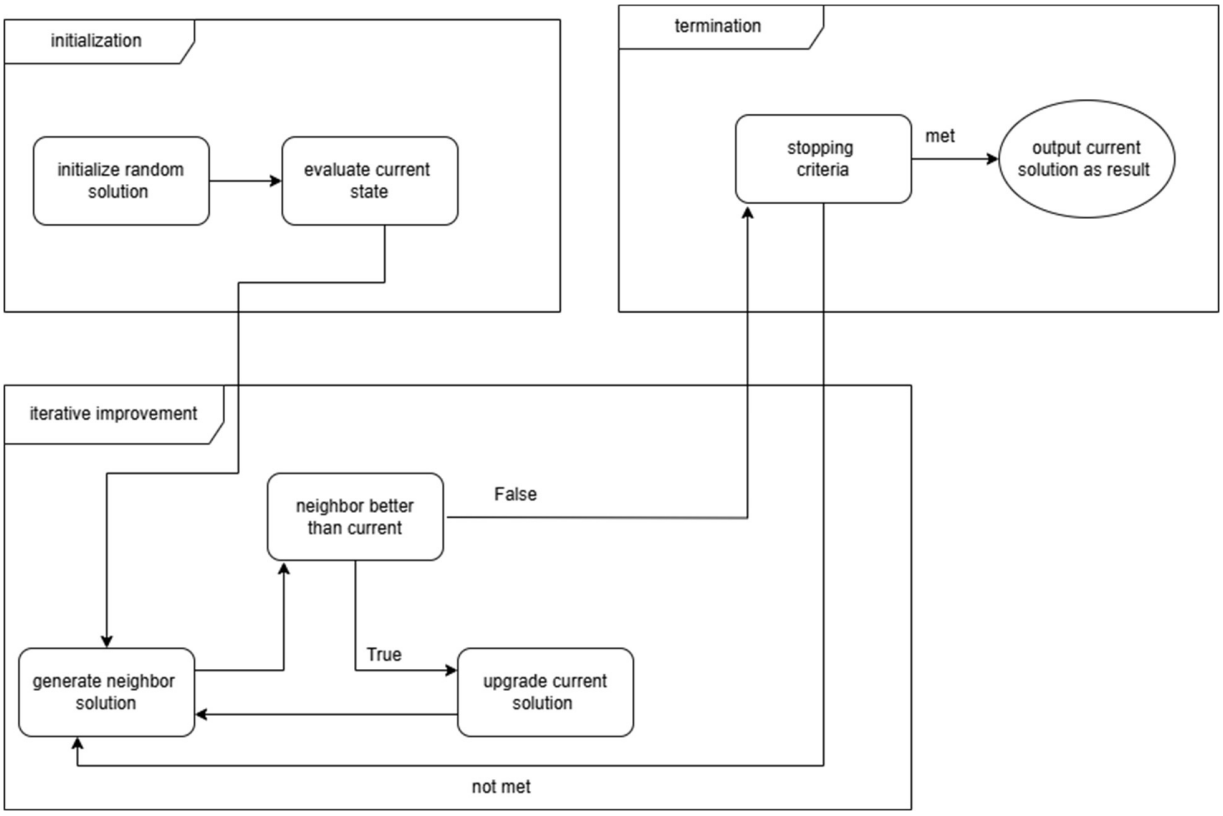
After the algorithm stuck at a local maximum solution the algorithm repeats its initial state once again but with different random state till it gets to the global maximum till its max restart completed.

```
for _ in range(max_restarts):
```

Flowchart:


Start
Initialize with random solution
Evaluate current solution
Generate neighboring solutions
Evaluate neighbors
No
Is there a better neighbor?
Yes
No
Stopping criteria met?
Replace current solution with best neighbor
Yes
Return best solution found

Block Diagram:



Literature Review:

Generalized Hill Climbing (GHC) algorithms provide a unified framework for describing a broad class of local search and metaheuristic optimization techniques used to solve hard discrete optimization problems. This paper presents a comprehensive analysis of GHC algorithms, focusing on their convergence properties, performance evaluation, and comparison with random restart local search. A key contribution is the discussion of the finite global visit probability as a practical performance metric that captures finite-time behavior more effectively than asymptotic convergence results. Theoretical convergence conditions are reviewed, with simulated annealing examined as a special case of the GHC framework. The paper concludes that while simulated annealing offers strong theoretical guarantees, random restart strategies and hybrid approaches often achieve superior performance in realistic computational settings.

1. Introduction

Discrete optimization problems appear in many real-world applications such as scheduling, network design, logistics, and machine learning. These problems often have large and complex search spaces, making exact solution methods computationally infeasible. As a result, heuristic and metaheuristic algorithms are widely used to obtain high-quality solutions within reasonable time limits. Generalized Hill Climbing (GHC) algorithms form a powerful class of such methods. They extend traditional hill climbing by incorporating probabilistic acceptance mechanisms that allow the search process to escape local optima. Well-known algorithms such as simulated annealing, threshold accepting, and the noising method can all be expressed within the GHC framework. This unification enables a consistent theoretical analysis of their convergence behavior and performance characteristics.

2.Review

Early work on local search and combinatorial optimization highlighted the difficulty of escaping local optima using purely deterministic methods. Simulated annealing, inspired by statistical physics, introduced stochastic acceptance rules and was shown to converge to global optima under carefully controlled cooling schedules. Hajek (1988) provided necessary and sufficient

conditions for the convergence of simulated annealing. Subsequent research explored alternative metaheuristics, including threshold accepting, tabu search, and random restart strategies. While many studies focused on asymptotic convergence, others argued that finite-time performance is more relevant in practice. The concept of random restart local search emerged as a simple yet effective approach, particularly for landscapes with many local optima.

3. Generalized Hill Climbing Framework

A GHC algorithm operates on a finite solution space equipped with an objective function and a neighborhood structure. At each iteration, a neighboring solution is randomly generated and accepted or rejected based on a hill-climbing random variable. This probabilistic rule allows occasional uphill moves, enabling exploration beyond local optima. The solution space can be partitioned into global optima, local optima, and non-optimal solutions. The neighborhood function defines the landscape structure and plays a critical role in determining algorithm performance. Reachability of the solution space is a key assumption for convergence analysis.

4. Finite Global Visit Probability

The finite global visit probability measures the likelihood that a GHC algorithm visits at least one global optimum within a finite number of macro iterations. A macro iteration represents a transition between local or global optima, possibly through non-optimal states. This metric provides a practical way to evaluate algorithm performance under limited computational budgets. Unlike asymptotic convergence results, it reflects the probability of success within a realistic execution horizon.

5. Convergence Analysis

The convergence of GHC algorithms can be characterized using necessary and sufficient conditions based on the probability of visiting global optima. In particular, convergence in probability requires that the cumulative probability of first reaching a global optimum diverges and that the algorithm increasingly remains at global optima once reached. Simulated annealing is shown to satisfy these conditions when its cooling schedule decreases slowly enough.

This establishes simulated annealing as a special case of the GHC framework and demonstrates the generality of the convergence theory.

6. Comparison with Random Restart Local Search

Random restart local search repeatedly applies deterministic local search from randomly selected initial solutions. Each restart corresponds to a macro iteration. This approach is particularly effective when the probability of falling into non-global local optima is high. Theoretical results indicate that random restart local search eventually outperforms non-convergent GHC algorithms. For convergent GHC algorithms, relative performance depends on neighborhood structure, cooling schedules, and the distribution of attraction basins.

7. Discussion

The analysis highlights the importance of designing effective neighborhood functions and parameter settings. Algorithms with strong asymptotic guarantees may still perform poorly in practice if convergence is too slow. Hybrid strategies that combine random restarts with stochastic local search often provide the best balance between exploration and exploitation.

References:

[1] S. H. Jacobson and E. Yücesan, “Analyzing the performance of generalized hill climbing algorithms,” Journal of Heuristics, vol. 10, pp. 387–405, 2004

4. Cultural:

A Cultural Algorithm (CA) is an evolutionary optimization algorithm similar to the Genetic Algorithm but with an extra intelligence layer called the "Belief Space."

It operates using two spaces:

- **Population Space: A set of candidate solutions → evolved each generation (just like GA).**
- **Belief Space:**

  This is the unique part:

  - **Stores knowledge extracted from the best individuals.**
  - **This knowledge guides future generations, making CA smarter and faster than GA.**

How It Works:

Step 1 — Initialize Population

```
population = [board.start.copy()] + [
[random.randint(0, n - 1) for _ in range(n)]
for _ in range(population_size - 1)]
```

create:

- **one initial state from the board**
- **many random states**

Step 2 — Initialize Belief Space

```
belief = board.start.copy()
```

Belief space starts with the initial board arrangement.

Step 3 — Evaluate & Sort by Fitness

population.sort(key=lambda s: heuristic(s, n))

best = population[0]

current_best = heuristic(best, n)

Lower heuristic = fewer queen conflicts.

Step 4 — Update Belief Space

This is the core of the algorithm:

top_half = population[:population_size // 2]

for i in range(n):

belief[i] = random.choice([s[i] for s in top_half])

Meaning:

- **Take the best 50% individuals**
- **For each column → pick a row from the top solutions**
- **This becomes the "knowledge" used for future generations**

Step 5 — Generate New Population Using Belief Space

idx = random.randint(0, n - 1)

child[idx] = belief[idx] if random.random() < 0.5 else random.randint(0, n - 1)

Children are guided by belief 50% of the time
and mutate randomly otherwise.

Step 6 — Detect Stagnation

If no improvement in 10 generations:

if stagnation_count >= 10:

refresh population

This helps escape local minima.

Step 7 — Save History for Plotting

history_best.append(current_best)

Used later to make a graph of improvement.

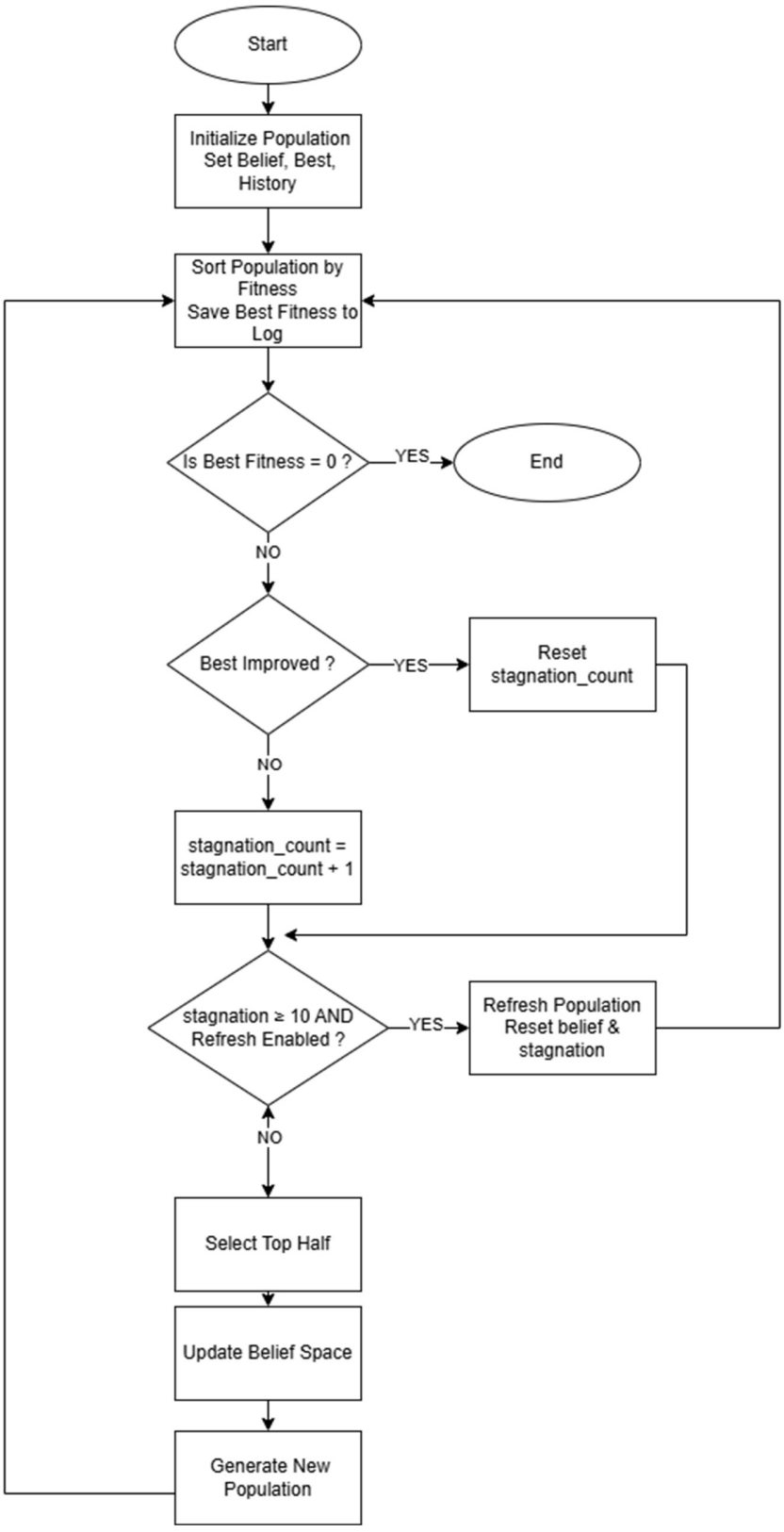
3. Difference Between Cultural Algorithm & Genetic Algorithm

Feature	Genetic Algorithm	Cultural Algorithm
Population	Maintains a population of candidate solutions	
Knowledge/ Belief Space	only through selection and crossover	*belief space* to store patterns from the best individuals
Selection	Choose parents based on fitness	Choose parents based on fitness (top half) and belief influence
Mutation	Random changes in offspring to maintain diversity	Random mutation combined with belief guidance
Global Search	via population diversity	via population diversity + belief space guidance
Local Search	Implicit, via selection and crossover	Explicit, via belief space improving individuals locally
Convergence Speed	slow if population is large	faster because belief space guides the search
Escaping Local Optima	via mutation or diversity	Better at escaping due to *belief + population + optional restarts*
Complexity	Depends on population size, generations, and N	Slightly higher due to sorting population + belief updates

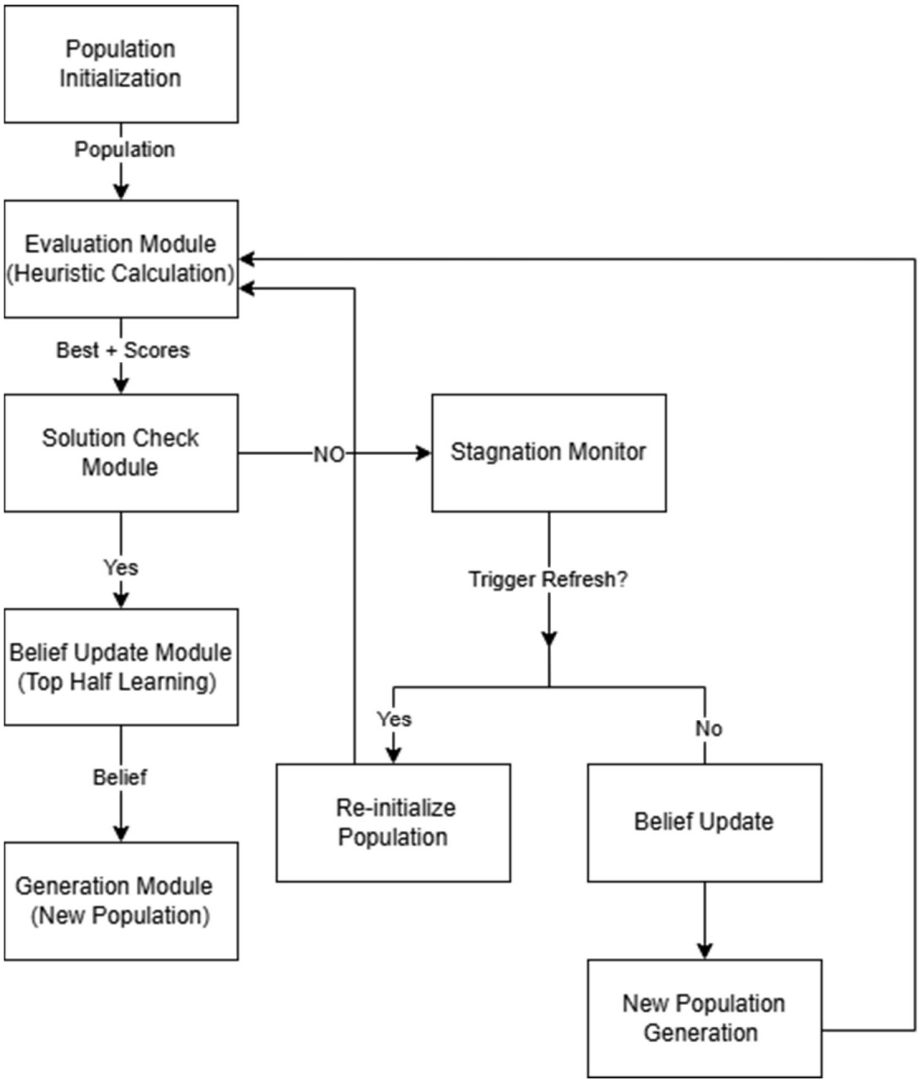
Crossover (Recombination):

Combines genes from two parents

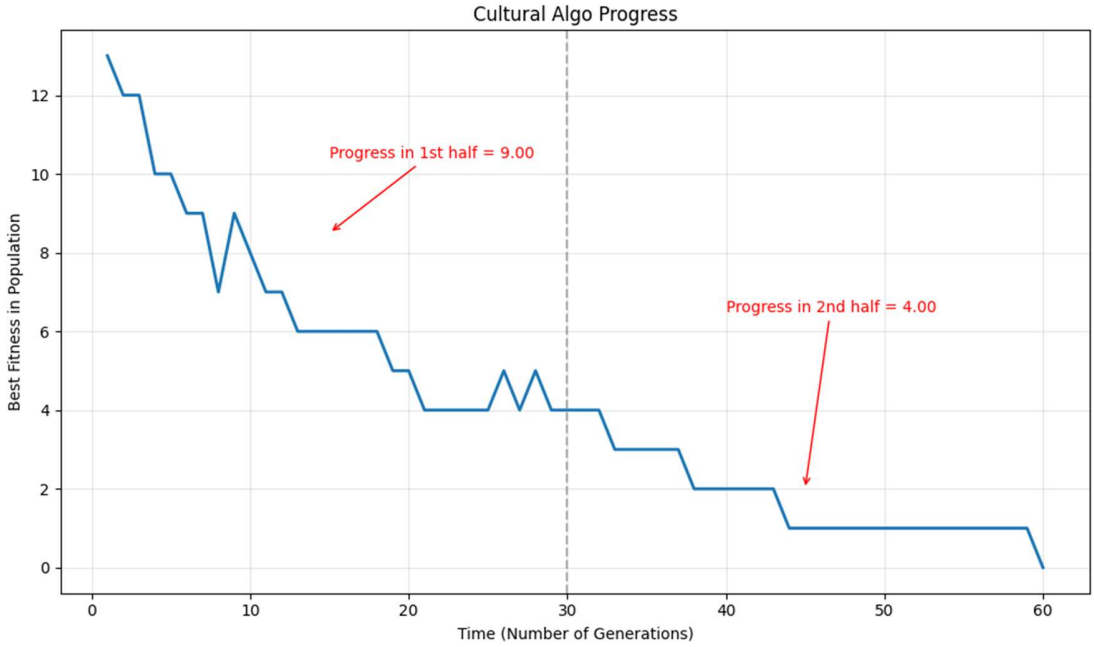
Flowchart:


Start
Initialize Population
Set Belief, Best,
History
Sort Population by
Fitness
Save Best Fitness to
Log
Is Best Fitness = 0 ?
YES
End
NO
Best Improved ?
YES
Reset
stagnation_count
NO
stagnation_count =
stagnation_count + 1
stagnation ≥ 10 AND
Refresh Enabled ?
YES
Refresh Population
Reset belief &
stagnation
NO
Select Top Half
Update Belief Space
Generate New
Population

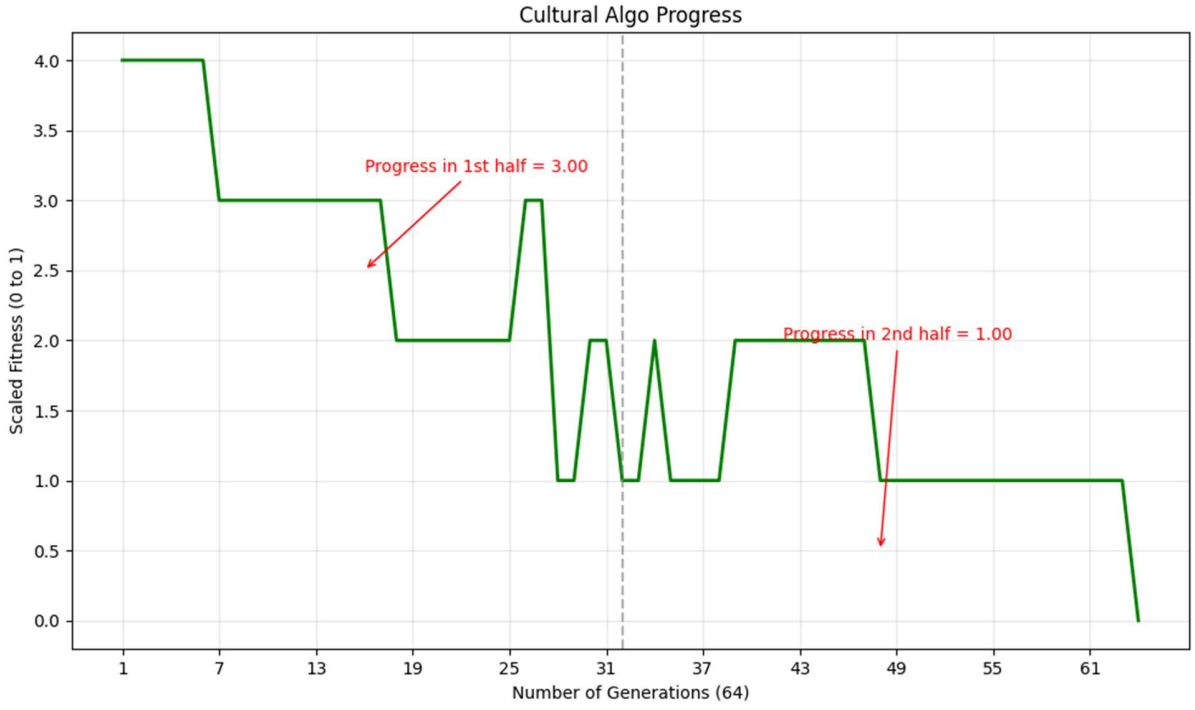
Block Diagram:



Cultural Progress Plot:



When n=20, 400 generation, and 200 population size



When n=11, 700 generation, and 110 population size

Literature Review:

Cultural Algorithms (CAs) are a class of computational intelligence methods inspired by the way human societies evolve knowledge over time. Unlike other swarm-based heuristics—such as Particle Swarm Optimization (PSO) or Ant Colony Optimization (ACO)—CAs use a dual inheritance system consisting of:

1. Population Space: Individuals explore the search space and generate candidate solutions.
2. Belief Space: A structured knowledge repository that stores cultural information learned from successful individuals.

The core idea is that the collective behaviour and experience of individuals form cultural knowledge, and this knowledge then guides individuals to improve future search efficiency.

Key Concepts in Cultural Algorithms:

- The belief space contains different types of knowledge (spatial, temporal, domain, normative, and situational) extracted over time.
- Individuals update the belief space, and the belief space influences individual behaviour creating a feedback loop.
- This cultural knowledge helps guide exploration, avoid premature convergence, and improve overall optimization capability.

Motivation for Cultural Algorithms:

Traditional swarm-based methods often:

- lack mechanisms to use all available information in the population,
- may prematurely converge to local optima (especially PSO),
- have difficulty maintaining diversity,
- struggle in multimodal, constrained, or dynamic optimization environments.

CAs address these problems by using organized and accumulated knowledge to:

- enhance exploration and exploitation,
- share diverse information among individuals,
- improve decision making across swarms or societies.

Applications Across Optimization Types:

The dissertation describes how cultural frameworks improve several optimization domains:

1. Single-objective optimization:
   Cultural knowledge helps individuals move more effectively within and across societies.
2. Multimodal optimization:
   Cultural knowledge facilitates communication among multiple swarms to avoid premature convergence.
3. Multiobjective optimization (MOPSO):
   Cultural information adapts each particle's momentum and acceleration individually, supporting polymorphic behavior and improving convergence to the Pareto front.
4. Constrained optimization:
   Cultural information guides multi-level PSO communication to satisfy constraints while optimizing the objective function.
5. Dynamic optimization:
   The belief space provides fast re-diversification and repulsion mechanisms after environmental changes, overcoming outdated memory and loss of diversity in PSO.

Overall Cultural Algorithms provide a powerful computational framework that:

- systematically organizes knowledge,
- enhances swarm-based optimization at multiple levels,
- supports sophisticated migration, repulsion, and adaptive parameter control,
- and improves performance across single-objective, multiobjective, constrained, and dynamic optimization problems.

References:

[1] Ali, M. Z., Alkhatib, K., & Tashtoush, Y. *Cultural algorithms: Emerging social structures for the solution of complex optimization problems*. International Journal of Artificial Intelligence, 11(13), 20–43.

[2] Daneshyari, M. (2010). *Cultural particle swarm optimization* (Doctoral dissertation, Oklahoma State University). Oklahoma State University Graduate College.

5. Heuristic Function:

Heuristic function is a **problem specific function** used in search algorithms to **estimate how close are you to reach goal state from current state**

It's a **guiding measure** to prioritize certain path over others

It's commonly referred to as **h(n)**, where n is current state

Use:

- Guide Search algorithms
- Reduce search space
- Provide way to score potential solutions

1)Heuristic1

```python
def heuristic1(state, n):
    conflicts = 0
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1
    return conflicts
```

h(n)= Number of attacking pairs

Logic:

- Iterate over all pairs of queens (i,j) with i<j
- Check if they are in same row or diagonal
- If either condition is true, increment conflicts
- Return number of total conflicting pairs

It counts pairs so each conflict is counted once

It has ***Time Complexity O(n^2)*** because of nested loop, so it's only good for small n

It has ***Space Complexity O(n)***

2)Heuristic2

```python
def heuristic2(state, n):
    row = [0] * n
    d1 = [0] * (2*n)
    d2 = [0] * (2*n)
    for c in range(n):
        r = state[c]
        row[r] += 1
        d1[c + r] += 1
        d2[c - r + n] += 1
    conflicts = 0
    for c in range(n):
        r = state[c]
        conflicts += (row[r] - 1)
        conflicts += (d1[c + r] - 1)
        conflicts += (d2[c - r + n] - 1)
    return conflicts
```

h(n)= Total number of conflicts for all queens

Logic:

- Initialize array to count, row[r] for number of queens in row r, d1[c+r] for queens in same major diagonal (top-left and bottom-left) and d1[c-r+n] for minor diagonal
- Loop over all columns c and update count for queens
- Loop again to get conflicts for each queen
- Return number of total conflicts

Optimized version by using arrays instead of nested loop

It has ***Time Complexity O(n)***

It has ***Space Complexity O(n)***

3)Heuristic3

```python
def heuristic3(state, n):
    conflicts = 0
    for c in range(n):
        r = state[c]
        for c2 in range(n):
            if c == c2:
                continue
            r2 = state[c2]
            if r == r2:
                conflicts += 1
            if abs(r - r2) == abs(c - c2):
                conflicts += 1
    return conflicts // 2
```

h(n)= Number of attacking pairs of queens

Logic:

- Loop each queen c and compare with every other queen c2
- If c==c2 then continue
- Check row for conflicts r==r2
- Check diagonals for conflicts abs(r-r2) == abs(c-c2)

- Divide conflicts by 2 at end so each conflict is only counted once

More readable and easier to comprehend

It has ***Time Complexity O(n^2)*** because of nested loop, so it's only good for small n

It has ***Space Complexity O(n)***

Comparison

All 3 heuristic aim to measure how bad board state is

Lower value-> fewer conflicts-> closer to solution

In terms of speed Heuristic2 is the fastest followed by Heuristic1 and Heuristic3

In terms of strength (how accurately it estimates true cost from current state to goal) Heuristic3 is the strongest as it counts exact tiles and is more accurate followed by Heuristic2 then Heuristic1

Results and Analysis

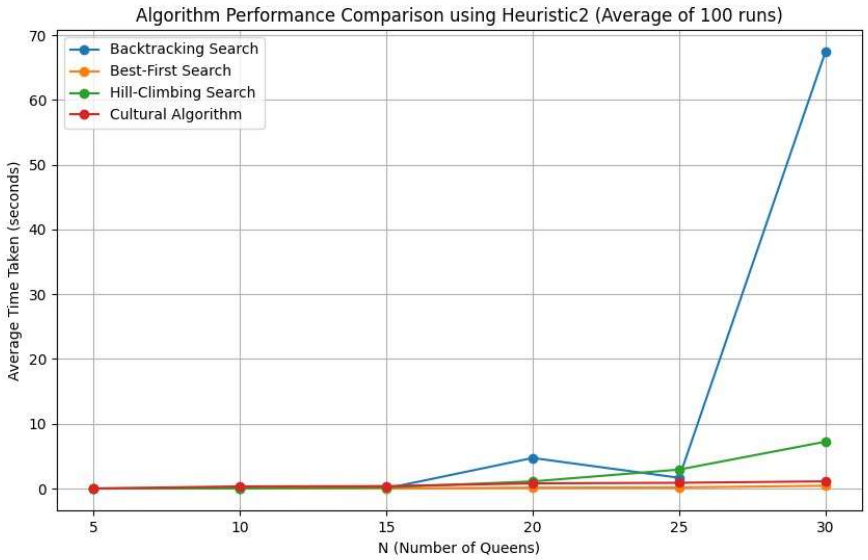
1. Results and Comparison

- To keep comparison fair, we used the same start state and same board size
- We ran each algorithm several times to get average
- For N=5

Algorithm	Avg Time	Success
Backtracking	0.000132	100%
Best-First	0.000302	100%
Hill-Climbing	0.000561	100%
Cultural	0.001600	100%

- For N=10

Algorithm	Avg Time	Success
Backtracking	4.754722	100%
Best-First	0.104976	100%
Hill-Climbing	1.153362	95%
Cultural	0.612077	100%



2. Analysis and Discussion

1)Backtracking:

- Always find solution if it exists
- Very efficient for small number of N
- Execution time increase exponentially as N increase because it searches exhaustively
- It's doesn't use heuristic so no guidance
- Not practical for large N

2)Best-First:

- Faster than Backtracking
- Explore less states due to using heuristic function to guide to the most promising node

- It depends on the heuristic function strength

3)Hill-Climbing:

- Works better with small N and higher success rate
- For large N it can get stuck in local minima
- Restarting improves success rate

4)Cultural:

- More stable than Hill-Climbing
- Faster than others for large N
- Performance improves as belief space increase
- Doesn't fall into local minima

Algorithm	Advantage	Disadvantage
Backtrack	Guaranteed solution	Slowest and exhaustive
Best-First	Fast	Depend on Heuristic
Hill-climbing	Faster than backtrack	Unreliable
Cultural	Fastest	Depend on population size and number of generations